

NP-Complete Proof Chain

Structured Exam Proofs for the Standard Reduction Path

How to Use These Notes

The Exam Pattern

Every NP-completeness proof has the same shape:

1. **Membership in NP:** state a certificate and say how to verify it in polynomial time.
2. **NP-hardness:** reduce a known **NP**-complete problem A to the target problem B .
3. **Correctness:** prove both directions:

$$x \in A \iff f(x) \in B.$$

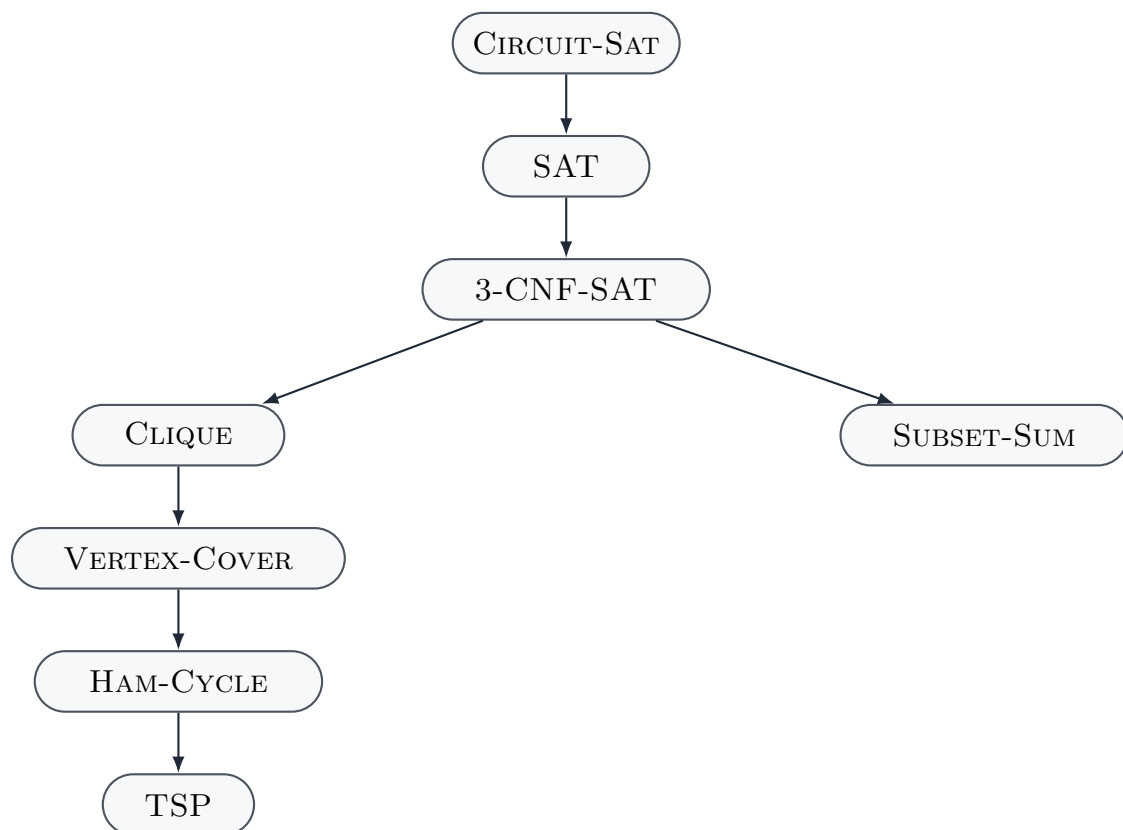
4. **Polynomial time:** state why the construction is polynomial.

Reduction Path to Memorize

CIRCUIT-SAT $\leq_p$ SAT $\leq_p$ 3-CNF-SAT $\leq_p$ CLIQUE $\leq_p$ VERTEX-COVER $\leq_p$ HAM-CYCLE $\leq_p$ TSP

and separately

3-CNF-SAT $\leq_p$ SUBSET-SUM.



1 Circuit-SAT is NP-Complete

Problem

$\text{CIRCUIT-SAT} = \{\langle C \rangle \mid C \text{ is a Boolean circuit with some input making } C \text{ output } 1\}.$

What to Remember

Any polynomial-time verifier can be unrolled into a polynomial-size circuit.

1. Membership in NP

A certificate is a truth assignment to the circuit input wires. The verifier evaluates each gate and checks whether the final output is 1. This takes time polynomial in the number of gates. Hence

$$\text{CIRCUIT-SAT} \in \mathbf{NP}.$$

2. NP-Hardness Reduction

Let L be any language in $\mathbf{NP}$. Then there is a polynomial-time verifier $A(x, y)$ such that

$$x \in L \iff \exists y A(x, y) = 1.$$

For a fixed input x , construct a circuit C_x whose free inputs are the certificate bits y and whose output is the result of simulating $A(x, y)$.

Because A runs in polynomial time, its computation has polynomially many steps and polynomial-size configurations. Unrolling these computation steps gives a polynomial-size circuit.

3. Correctness

$$x \in L \iff \exists y A(x, y) = 1 \iff \exists y C_x(y) = 1 \iff C_x \in \text{CIRCUIT-SAT}.$$

Thus every language in $\mathbf{NP}$ reduces to CIRCUIT-SAT, so CIRCUIT-SAT is $\mathbf{NP}$ -hard. Since $\text{CIRCUIT-SAT} \in \mathbf{NP}$, CIRCUIT-SAT is $\mathbf{NP}$ -complete.

2 SAT is NP-Complete

Problem

$\text{SAT} = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}.$

Reduction Used

$$\text{CIRCUIT-SAT} \leq_p \text{SAT}.$$

Replace circuit wires by formula variables and enforce every gate by a local formula constraint.

1. Membership in NP

A certificate is a truth assignment to the variables of ϕ . The verifier evaluates ϕ under this assignment and accepts iff the value is 1. Hence

$$\text{SAT} \in \mathbf{NP}.$$

2. Construction

Given a circuit C , create one formula variable x_w for every wire w . For each gate, add a constraint saying that the output wire equals the correct function of the input wires:

$$x_z \Leftrightarrow (x_a \wedge x_b), \quad x_z \Leftrightarrow (x_a \vee x_b), \quad x_z \Leftrightarrow \neg x_a.$$

If x_{out} is the output-wire variable, define

$$\phi_C = x_{\text{out}} \wedge \bigwedge_{\text{gates } g} \text{Constraint}(g).$$

The formula has size polynomial in the circuit size.

3. Correctness

If C is satisfiable, then a satisfying input gives a definite value to every wire. Assign each x_w that wire value. Then all gate constraints are true and $x_{\text{out}} = 1$, so ϕ_C is satisfiable.

Conversely, if ϕ_C is satisfiable, the gate constraints describe a consistent circuit evaluation and force $x_{\text{out}} = 1$. Therefore the input-wire variables give a satisfying input for C .

Thus

$$C \in \text{CIRCUIT-SAT} \iff \phi_C \in \text{SAT}.$$

Since CIRCUIT-SAT is **NP**-complete and $\text{SAT} \in \text{NP}$, SAT is **NP**-complete.

3 3-CNF-SAT is NP-Complete

Problem

3-CNF-SAT is the set of satisfiable CNF formulas in which every clause has exactly three distinct literals.

Reduction Used

$$\text{SAT} \leq_p \text{3-CNF-SAT}.$$

Use parse-tree variables, convert local constraints to CNF, and pad short clauses.

1. Membership in NP

A certificate is a truth assignment. The verifier checks all 3-literal clauses and accepts iff every clause is true. Hence

$$\text{3-CNF-SAT} \in \text{NP}.$$

2. Construction

Given a formula ϕ , build its binary parse tree. Introduce a new variable y_i for the value of each internal node, and let y_r be the root variable. Add local constraints G_i saying that each y_i correctly represents the connective at that node:

$$\phi' = y_r \wedge \bigwedge_i G_i.$$

Each G_i uses at most three variables.

Convert each G_i to CNF by its truth table. Since it has at most three variables, its truth table has at most $2^3 = 8$ rows, so this creates only a constant number of clauses.

Finally, replace short clauses by equivalent 3-literal clauses:

$$(\ell_1 \vee \ell_2) \mapsto (\ell_1 \vee \ell_2 \vee p) \wedge (\ell_1 \vee \ell_2 \vee \neg p),$$

$$(\ell) \mapsto (\ell \vee p \vee q) \wedge (\ell \vee p \vee \neg q) \\ \wedge (\ell \vee \neg p \vee q) \wedge (\ell \vee \neg p \vee \neg q).$$

3. Correctness and Size

The parse-tree constraints preserve satisfiability, the truth-table conversion is logically equivalent for each local constraint, and the padding rules are equivalent to the original short clauses. Therefore the final 3-CNF formula ψ satisfies

$$\phi \in \text{SAT} \iff \psi \in \text{3-CNF-SAT}.$$

Only constantly many clauses are added per connective, so the construction is polynomial. Since SAT is **NP**-complete and $\text{3-CNF-SAT} \in \text{NP}$, 3-CNF-SAT is **NP**-complete.

4 Clique is NP-Complete

Problem

$$\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ has a clique of size at least } k\}.$$

Reduction Used

$$\text{3-CNF-SAT} \leq_p \text{CLIQUE}.$$

Make one vertex for each literal occurrence. A clique chooses one consistent literal from every clause.

1. Membership in NP

A certificate is a set S of k vertices. The verifier checks that every pair of vertices in S is joined by an edge. Hence

$$\text{CLIQUE} \in \text{NP}.$$

2. Construction

Let

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

be a 3-CNF formula. For each literal occurrence in each clause, create one vertex. Put an edge between two vertices iff:

- the two vertices come from different clauses, and
- their literals are consistent, meaning one is not the negation of the other.

Output $\langle G, m \rangle$.

3. Correctness

If ϕ is satisfiable, choose one true literal from each clause. These m vertices are from different clauses and are pairwise consistent, so they form a clique of size m .

Conversely, suppose G has a clique of size m . Since vertices from the same clause are not adjacent, the clique contains exactly one vertex from each clause. Since clique vertices are pairwise consistent, we can assign the chosen literals to be true. Then every clause has a true literal, so ϕ is satisfiable. Thus

$$\phi \in \text{3-CNF-SAT} \iff \langle G, m \rangle \in \text{CLIQUE}.$$

The graph is polynomial size. Since 3-CNF-SAT is **NP**-complete and $\text{CLIQUE} \in \text{NP}$, CLIQUE is

NP-complete.

5 Vertex-Cover is NP-Complete

Problem

$\text{VERTEX-COVER} = \{\langle G, k \rangle \mid G \text{ has a vertex cover of size at most } k\}.$

Reduction Used

$\text{CLIQUE} \leq_p \text{VERTEX-COVER}.$

Use the complement graph. A clique of size k becomes a vertex cover of size $|V| - k$.

1. Membership in NP

A certificate is a set $C \subseteq V(G)$ with $|C| \leq k$. The verifier checks that every edge has at least one endpoint in C . Hence

$\text{VERTEX-COVER} \in \mathbf{NP}.$

2. Construction

Given $\langle G, k \rangle$ for CLIQUE, construct the complement graph $\overline{G}$:

$$(u, v) \in E(\overline{G}) \iff u \neq v \text{ and } (u, v) \notin E(G).$$

Output

$$\langle \overline{G}, |V| - k \rangle.$$

3. Correctness

If G has a clique S of size k , then $V - S$ is a vertex cover in $\overline{G}$. No edge of $\overline{G}$ can have both endpoints in S , because all pairs in S are edges of G . Hence every edge of $\overline{G}$ touches $V - S$. Conversely, if $\overline{G}$ has a vertex cover C with $|C| \leq |V| - k$, then $V - C$ has size at least k . Since C covers every edge of $\overline{G}$, no two vertices of $V - C$ are adjacent in $\overline{G}$. Therefore every pair in $V - C$ is adjacent in G , so G has a clique of size at least k .

Thus

$$\langle G, k \rangle \in \text{CLIQUE} \iff \langle \overline{G}, |V| - k \rangle \in \text{VERTEX-COVER}.$$

The complement is computable in polynomial time. Since CLIQUE is **NP**-complete and $\text{VERTEX-COVER} \in \mathbf{NP}$, VERTEX-COVER is **NP**-complete.

6 Hamiltonian Cycle is NP-Complete

Problem

$\text{HAM-CYCLE} = \{\langle G \rangle \mid G \text{ has a Hamiltonian cycle}\}.$

Reduction Used

$\text{VERTEX-COVER} \leq_p \text{HAM-CYCLE}.$

Use one edge-widget per edge of the original graph and k selector vertices. The selectors choose the

k vertices of the cover.

1. Membership in NP

A certificate is an ordering of all vertices. The verifier checks that each vertex appears once and that consecutive vertices, including the last and first, are joined by edges. Hence

$$\text{HAM-CYCLE} \in \mathbf{NP}.$$

2. Preprocessing

Given a vertex-cover instance (G, k) , delete isolated vertices. If $E(G)$ is empty, output a fixed yes-instance for HAM-CYCLE, such as a triangle. Otherwise replace k by $\min(k, |V|)$.

After deleting isolated vertices, G has a vertex cover of size at most k iff it has one of size exactly k , because any smaller cover can be padded with extra vertices.

3. Edge Widget

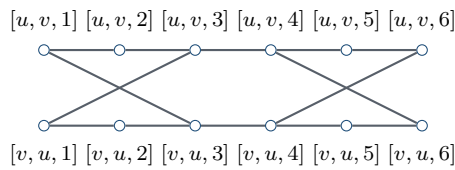
For each edge $\{u, v\} \in E(G)$, create a 12-vertex widget W_{uv} with vertices

$$[u, v, 1], \dots, [u, v, 6] \quad \text{and} \quad [v, u, 1], \dots, [v, u, 6].$$

Only the four corner vertices

$$[u, v, 1], [u, v, 6], [v, u, 1], [v, u, 6]$$

connect to the outside.



4. Widget Behavior

The widget can be traversed in exactly the useful ways:

- through the u side while visiting all widget vertices;
- through the v side while visiting all widget vertices;
- through two disjoint straight paths, one on the u side and one on the v side.

These cases represent the edge $\{u, v\}$ being covered by u , by v , or by both.

5. Full Construction

For each vertex $u \in V(G)$, order its neighbors as

$$u^{(1)}, u^{(2)}, \dots, u^{(d(u))}.$$

Connect the u -sides of the widgets incident on u into one chain:

$$([u, u^{(i)}, 6], [u, u^{(i+1)}, 1]) \quad \text{for } i = 1, \dots, d(u) - 1.$$

Add selector vertices $s_1, \dots, s_k$. Connect every selector to the first and last vertex of every vertex-chain:

$$s_j \text{ adjacent to } [u, u^{(1)}, 1] \quad \text{and} \quad [u, u^{(d(u))}, 6].$$

The graph G' has polynomial size.

6. Correctness

If G has a vertex cover $C = \{u_1, \dots, u_k\}$, form a Hamiltonian cycle in G' by going

$$s_1 \rightarrow \text{chain of } u_1 \rightarrow s_2 \rightarrow \text{chain of } u_2 \rightarrow \dots \rightarrow s_k \rightarrow \text{chain of } u_k \rightarrow s_1.$$

Inside each edge-widget, use the traversal corresponding to whether that edge is covered by one selected endpoint or by both. Since C covers every edge, every widget is fully visited.

Conversely, suppose G' has a Hamiltonian cycle. Break the cycle into maximal paths between selector vertices. Each such path runs through one vertex-chain, so it selects one vertex of G . There are k selectors, so at most k vertices are selected. Since every widget must be visited, every original edge has at least one selected endpoint. Hence the selected vertices form a vertex cover.

Thus

$$\langle G, k \rangle \in \text{VERTEX-COVER} \iff \langle G' \rangle \in \text{HAM-CYCLE}.$$

Since VERTEX-COVER is NP-complete and HAM-CYCLE $\in$ NP, HAM-CYCLE is NP-complete.

For Exams

This is the one proof where a picture matters. Draw the widget, state the three traversal cases, then explain selector vertices choose the cover.

7 Traveling-Salesman Problem is NP-Complete

Problem

$$\text{TSP} = \{\langle G, c, B \rangle \mid G \text{ is complete and has a tour of cost at most } B\}.$$

Reduction Used

$$\text{HAM-CYCLE} \leq_p \text{TSP}.$$

Cost 1 for original edges, cost 2 for nonedges, and bound n .

1. Membership in NP

A certificate is an ordering of all vertices. The verifier checks that it is a tour, sums the edge costs, and checks whether the sum is at most B . Hence

$$\text{TSP} \in \text{NP}.$$

2. Construction

Given an undirected graph $G = (V, E)$ with $n = |V|$, construct the complete graph $G' = (V, E')$ on the same vertices. Define

$$c(u, v) = \begin{cases} 1, & \text{if } (u, v) \in E, \\ 2, & \text{if } (u, v) \notin E. \end{cases}$$

Set the cost bound

$$B = n.$$

3. Correctness

If G has a Hamiltonian cycle, then the same cycle is a TSP tour in G' using only cost-1 edges, so its total cost is n .

Conversely, suppose G' has a tour of cost at most n . Every tour uses exactly n edges, and each edge has cost at least 1. Therefore every edge in the tour must have cost exactly 1, so every tour

edge belongs to E . Hence the tour is a Hamiltonian cycle in G .

Thus

$$G \in \text{HAM-CYCLE} \iff \langle G', c, n \rangle \in \text{TSP}.$$

The construction is polynomial. Since HAM-CYCLE is **NP**-complete and $\text{TSP} \in \text{NP}$, TSP is **NP**-complete.

8 Subset-Sum is NP-Complete

Problem

$$\text{SUBSET-SUM} = \{ \langle S, t \rangle \mid \text{some subset of integers in } S \text{ sums to } t \}.$$

Reduction Used

$$3\text{-CNF-SAT} \leq_p \text{SUBSET-SUM}.$$

Use digit columns to force one truth value per variable and at least one true literal per clause.

1. Membership in NP

A certificate is the chosen subset. The verifier sums its elements and checks whether the sum equals the target. Hence

$$\text{SUBSET-SUM} \in \text{NP}.$$

2. Construction

Let

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

be a 3-CNF formula with variables $x_1, \dots, x_n$. Build numbers with $n + m$ digit columns:

- one column for each variable;
- one column for each clause.

For each variable x_i , create two numbers T_i and F_i :

- both have digit 1 in variable column i and 0 in all other variable columns;
- T_i has digit 1 in clause column j iff x_i appears in C_j ;
- F_i has digit 1 in clause column j iff $\neg x_i$ appears in C_j .

For each clause C_j , add slack numbers A_j and B_j with digits 1 and 2 in clause column j , and zeros elsewhere.

The target t has digit 1 in every variable column and digit 4 in every clause column. Use base 10, so no carries occur.

3. Correctness

If ϕ is satisfiable, choose T_i when $x_i = 1$ and F_i when $x_i = 0$. Then each variable column sums to 1. In a clause column, the chosen variable numbers contribute 1, 2, or 3 because at least one literal is true. Use slack digits to bring the sum to 4:

$$1 + 1 + 2 = 4, \quad 2 + 2 = 4, \quad 3 + 1 = 4.$$

Thus the chosen subset sums to t .

Conversely, suppose a subset sums to t . Since the target has digit 1 in each variable column, exactly one of T_i, F_i is chosen for each variable. This defines a truth assignment. In each clause column, the two slack numbers can contribute at most 3, while the target is 4. Therefore at least one chosen

variable number must contribute 1 in that clause column, meaning at least one literal in the clause is true.

Therefore

$$\phi \in \text{3-CNF-SAT} \iff \langle S, t \rangle \in \text{SUBSET-SUM}.$$

The construction has polynomially many numbers and digits. Since 3-CNF-SAT is **NP**-complete and $\text{SUBSET-SUM} \in \text{NP}$, SUBSET-SUM is **NP**-complete.

Final Memory Sheet

One-Line Reduction Ideas

Problem	What to write for NP-hardness
CIRCUIT-SAT	Simulate any NP verifier by a polynomial-size circuit.
SAT	Replace each circuit wire by a formula variable and add gate constraints.
3-CNF-SAT	Parse-tree variables, truth-table CNF for local constraints, and padding.
CLIQUE	One vertex per literal occurrence; edges join consistent literals from different clauses.
VERTEX-COVER	Complement graph: clique of size k becomes cover of size $n - k$.
HAM-CYCLE	Edge widgets plus k selectors encode choosing k cover vertices.
TSP	Cost 1 for original edges, cost 2 for nonedges, bound n .
SUBSET-SUM	Digit columns force one truth value per variable and one true literal per clause.

Fast Exam Checklist

For each proof, do not forget:

- certificate for membership in **NP**;
- source problem for the reduction;
- construction of the new instance;
- both directions of correctness;
- polynomial-time size bound.